

ds. Select the *Type* field to *reg* and set *Default Value* to 0, and *Visibility* as *Yes*, as shown in Fig. 7.

You can switch over to *States* and *Transitions* tabs to see the values. Change the *Visibility* field in the *Transitions* tab to *Only non-default*, to make the 1 equation not show up on the diagram.

Note: *rd* and *ds* now appear as state attributes in *States* tab. This means you will be able to assign particular values to them in particular states. *Transitions* tab does not show *rd* and *ds* because it makes no sense to define registered outputs on a transition.

Individual state attributes. Now the outputs appear on the states with a *<=* after them as shown in Fig. 8. This indicates registered outputs (= means combinational).

To enter the non-default values for *rd* and *ds*, right click on the *Read* state and select *Edit State Properties* to bring up the menu, or just double click the *Read* state bubble. Change the value of

rd to 1. Repeat this for the other states to add appropriate output values (*rd* = 1 in *DLI*, *ds* = 1 in *Done*).

Individual transition attributes. Double click on the *Idle* to *Read* transition to bring up the transition menu. Change the equation to *go*. Repeat this for the state transition from *DLI* back to *Read* that has an equation of *us*. Our final state diagram looks like Fig. 9, a Cliff Cummings' basic FSM.

Output

Now we can run the backend and generate code using:

```
fizzim.pl < cliff.fzm > cliff.v
```

(The default encoding is heros)

The default technique does not really allow for the *default_state_is_x* behaviour, so the output looks rather different when this attribute is set (refer to code generated). For *implied_loopback* as set assumption (setting either flag is not recommended as it can result in inferred latches), you can take output using *onehot* when *implied_loopback*

is set by using:

```
fizzim.pl -enc onehot < cliff.fzm >  
cliff.v
```

Many designers prefer to design their FSMs as a state diagram (bubbles and arrows) and then manually translate this diagram into Verilog. For these designers, it would certainly be handy to design the FSM directly in a graphical tool and allow the software to generate the Verilog code.

We have discussed the basic operation of this tool whose simple and easy-to-use features, with a familiar Windows look and feel of GUI, clearly indicate that it is a handy tool for designing FSMs. The main feature which sets it apart is that its GUI is independent of Verilog generation, which allows for easy modification of the Verilog output that can be the main concern of many designers for better understanding and designing of FSMs. ●

The author is a technical journalist at EFY, Gurgaon

DO YOU HAVE EXCITING SOFTWARE FOR ELECTRONICS DESIGN ENGINEERS?

Get it bundled with the
EFY PLUS DVD

To know more: contact Atul (+91-8800094211)
or write to us at efyenq@efyindia.com

